GUIDE COMPLET

Sécurité Web • Checklist Idée • Défis d'Ingénierie

Référence permanente pour développeur (Junior → Senior)
Tout ce que tu dois connaître, respecter et appliquer
pour construire, sécuriser et faire évoluer une application solide.

Niveaux	Junior → Intermédiaire → Senior / Pro
Objectif	Rester capable de sécuriser, maintenir et faire évoluer soi-même
Contenu	Sécurité + Checklist avant de démarrer + 10 défis critiques + Rate limiting
À utiliser pour	JiaKo, Kofira et tout futur projet web ou mobile

Règle d'or : La sécurité et la solidité technique ne sont pas des fonctionnalités qu'on ajoute à la fin.
Ce sont des contraintes présentes dès le premier jour.

Document de référence – À relire régulièrement

PARTIE 1 — GUIDE DE SÉCURITÉ WEB

1. Pourquoi la sécurité est non négociable

Une application web qui stocke des comptes, des données personnelles ou qui expose des liens publics est une cible. Les attaques ne sont pas théoriques : bots, vol de comptes, spam, injection de code, vol de données, abus de fonctionnalités. Si tu négliges la sécurité, tu risques la perte de confiance, des fuites de données et des problèmes légaux (notamment RGPD).

Règle d'or : La sécurité n'est pas une fonctionnalité qu'on ajoute à la fin. C'est une contrainte qui doit être présente dès le premier jour.

2. Les trois niveaux de maîtrise

NIVEAU JUNIOR / DÉBUTANT

Objectif : Éviter les erreurs les plus graves et les plus fréquentes.

- **Mots de passe** → Ne jamais les stocker en clair. Toujours les hasher (bcrypt ou argon2).
- **HTTPS** → Obligatoire. L'adresse doit commencer par https:// (Vercel le fait automatiquement).
- **Secrets** → Aucune clé API, mot de passe de base de données ou secret ne doit apparaître dans le code frontend ou dans Git.
- **Validation côté serveur** → Ne jamais faire confiance à ce que le navigateur envoie. Tout doit être re-vérifié côté serveur.
- **Permissions** → Un utilisateur ne doit pouvoir accéder qu'à ses propres données. Vérifier systématiquement.
- **Variables d'environnement** → Tous les secrets dans des fichiers .env (jamais commités).

Exemple d'erreur junior classique : quelqu'un change l'ID dans l'URL (/etudes/123 → /etudes/456) et voit les données d'un autre utilisateur (IDOR).

NIVEAU INTERMÉDIAIRE

Objectif : Protéger activement contre les abus et les attaques courantes.

- **Rate limiting** → Limiter le nombre de requêtes (login, formulaires, API) par utilisateur / IP / fenêtre de temps.
- **Protection XSS** → Échapper tout contenu affiché qui vient d'un utilisateur. Utiliser une Content Security Policy (CSP).
- **Protection CSRF** → Tokens anti-CSRF ou cookies SameSite.
- **Injection SQL** → Toujours utiliser un ORM ou des requêtes paramétrées. Jamais de concaténation de chaînes.
- **Gestion des sessions** → Cookies httpOnly + Secure, durée de vie limitée, invalidation au logout.
- **Logs et monitoring** → Savoir détecter les comportements anormaux (trop de tentatives, trop de créations…).
- **Row Level Security (RLS)** → Si tu utilises Supabase/PostgreSQL, activer RLS pour que la base elle-même refuse les accès non autorisés.

NIVEAU SENIOR / PRO

Objectif : Anticiper, modéliser les menaces et construire une application résiliente.

- **Threat modeling** → Identifier les surfaces d'attaque avant de coder (qui peut faire quoi, avec quelles données).
- **Idempotence** → Les opérations critiques (paiement, création de ressource) doivent pouvoir être rejouées sans effet de bord.
- **Atomicité et transactions** → Soit tout réussit, soit tout échoue. Pas d'état intermédiaire incohérent.
- **Zero-downtime** → Migrations et déploiements qui ne cassent pas l'application en production.
- **Cohérence distribuée** → Comprendre les compromis (CAP) quand plusieurs services ou bases sont impliqués.
- **Audit et traçabilité** → Savoir qui a fait quoi et quand (surtout pour les actions sensibles).
- **Défense en profondeur** → Plusieurs couches de protection (frontend + backend + base + reverse proxy).

3. Les attaques les plus courantes à connaître

Attaque	Ce que c'est	Comment se protéger
XSS	Injection de code JavaScript malveillant	Échapper les données affichées + CSP
CSRF	Forcer une action à l'insu de l'utilisateur	Tokens anti-CSRF / SameSite cookies
Injection SQL	Modifier les requêtes base de données	ORM ou requêtes paramétrées
Brute force	Tester des milliers de mots de passe	Rate limiting + blocage temporaire

IDOR	Accéder aux données d'un autre en changeant un ID	Vérifier les permissions à chaque requête
Spam / Bots	Remplir des formulaires en masse	Rate limiting + honeypot / captcha
Vol de session	Récupérer le token de connexion	httpOnly + Secure cookies, durée limitée
DoS / DDoS	Saturer le serveur avec trop de requêtes	Rate limiting + reverse proxy + CDN

4. Rate Limiting – La protection essentielle contre les abus

Le rate limiting consiste à limiter le nombre d'actions qu'un utilisateur, une IP ou un appareil peut effectuer dans une fenêtre de temps donnée. Exemples concrets : 5 tentatives de connexion par minute, 10 créations de formulaire par heure, 100 requêtes API par minute.

Sans rate limiting, un bot peut envoyer 10 000 requêtes par seconde et saturer (ou faire planter) ton application. C'est l'une des premières défenses contre le spam, le brute-force et les attaques DoS.

Où l'appliquer en priorité : login, inscription, formulaires publics, endpoints d'API sensibles, génération de liens publics.

Solution classique : un reverse proxy (Nginx, Cloudflare, Vercel Edge Middleware, ou bibliothèque comme rate-limiter-flexible) qui compte les requêtes et bloque au-delà du seuil.

5. Comment travailler avec l'IA (vibe coding sécurisé)

Tu restes le décideur. L'IA propose, tu valides. Phrase type à ajouter dans tes prompts :

« Applique les bonnes pratiques de sécurité : validation côté serveur, vérification des permissions, protection contre les abus (rate limiting), idempotence si pertinent. Explique-moi clairement ce que tu as mis en place et pourquoi. »

Ne fais jamais confiance aveuglément. Demande toujours : « Quelles sont les surfaces d'attaque de cette fonctionnalité ? »

6. Posture de travail recommandée

Tu travailles en mode **Junior exigeant de niveau Intermédiaire**. Tu restes humble (tu ne sais pas encore tout), mais tu refuses les réponses vagues. Tu forces l'IA à justifier ses choix techniques et de sécurité. Tu restes le décideur final.

Ce qui ne disparaît jamais, même en senior :

- Git / versionning propre (commits clairs, branches)
- Réfléchir avant de coder (clarifier le besoin)
- Vérifier la sécurité des parties critiques
- Documenter les décisions importantes
- Tester les cas limites (URL modifiée, données invalides, spam, double-clic…)

7. Résumé – Ce que tu dois faire concrètement

À chaque nouvelle fonctionnalité :

1. Demander à l'IA les mesures de sécurité associées
2. Vérifier que les permissions sont contrôlées côté serveur
3. S'assurer qu'aucun secret n'est exposé
4. Ajouter du rate limiting si l'action peut être abusée
5. Tester les cas limites (URL modifiée, données invalides, spam, double soumission…)
6. Commiter avec un message clair

Phrase à retenir : « Je ne fais pas confiance au frontend. Tout ce qui est important est vérifié et protégé côté serveur. »

PARTIE 2 — LES 10 DÉFIS D'INGÉNIERIE LOGICIELLE CRITIQUES

Ces défis apparaissent dans presque toutes les applications réelles. Les connaître te permet d'anticiper les problèmes au lieu de les découvrir en production. Même si tu codes avec de l'IA, tu dois savoir les reconnaître et exiger les bonnes solutions.

01 — Concurrence / Race conditions (le dernier ticket)

Problème : Deux utilisateurs cliquent en même temps sur le dernier ticket / la dernière place. Les deux paient, mais un seul devrait obtenir la ressource.

Risque : Survente, double attribution, perte de confiance.

Ce qu'il faut faire : Verrouillage optimiste ou pessimiste, transactions atomiques, file d'attente, ou « claim » avec expiration. Ne jamais faire confiance à un simple « if stock > 0 ».

02 — Idempotence (le paiement fantôme)

Problème : L'utilisateur clique deux fois sur « Payer », le réseau timeout, le client réessaie… et le compte est débité deux fois.

Risque : Double facturation, plaintes, litiges.

Ce qu'il faut faire : Chaque opération critique doit avoir une clé d'idempotence (idempotency key). Si la même clé arrive une seconde fois, on retourne le résultat déjà connu sans re-exécuter.

03 — Résilience & atomicité (crash pendant un transfert)

Problème : Pendant un transfert d'argent ou une opération multi-étapes, la base de données ou le serveur plante au milieu.

Risque : État incohérent : argent débité d'un côté mais pas crédité de l'autre.

Ce qu'il faut faire : Transactions ACID, patterns Saga / Outbox, compensation (rollback métier), ou at-least-once + idempotence. Toujours prévoir le « que se passe-t-il si ça plante ici ? ».

04 — Scalabilité (pic de 1 million d'utilisateurs)

Problème : L'application fonctionne bien avec 1 000 utilisateurs, puis un pic soudain (pub, viralité) multiplie le trafic par 100 ou 1000.

Risque : CPU / base saturés, timeouts, site inaccessible.

Ce qu'il faut faire : Architecture capable de scale horizontal (stateless), cache, CDN, rate limiting, files d'attente, et tests de charge avant le lancement. Anticiper les goulots d'étranglement (DB souvent le premier).

05 — Autorisation & IDOR (compte admin / changement d'ID)

Problème : L'utilisateur change un ID dans l'URL ou dans une requête API et accède aux données d'un autre (ou à des fonctions admin).

Risque : Fuite de données personnelles, actions non autorisées.

Ce qu'il faut faire : Vérifier à CHAQUE requête que l'utilisateur authentifié a bien le droit d'accéder à cette ressource précise. Ne jamais se contenter de « l'utilisateur est connecté ».

06 — Cohérence multi-services (crash pendant une commande)

Problème : Une commande implique paiement + stock + création de commande. Un des services plante au milieu.

Risque : Commande payée mais stock non décrémenté, ou inverse.

Ce qu'il faut faire : Éviter les transactions distribuées fragiles. Préférer des patterns : orchestrateur, Saga, Outbox, messages idempotents. Accepter une cohérence éventuelle plutôt qu'une atomicité impossible.

07 — Déploiement sans downtime & migrations

Problème : Tu changes le schéma de la base ou déploies une nouvelle version pendant que des utilisateurs sont connectés.

Risque : Erreurs 500, données corrompues, besoin de rollback urgent.

Ce qu'il faut faire : Migrations backward-compatible (ajouter d'abord, supprimer plus tard), dual-write si nécessaire, feature flags, blue/green ou rolling deploy. Toujours pouvoir revenir en arrière.

08 — Synchronisation offline & conflits

Problème : L'utilisateur modifie les mêmes données sur deux appareils (ou en mode offline), puis les deux versions se synchronisent.

Risque : Perte de données, écrasement silencieux d'une version.

Ce qu'il faut faire : Stratégie de résolution de conflits claire (last-write-wins, merge, ou intervention utilisateur). Timestamps, versioning (vector clocks ou simple version number), et tests de scénarios de conflit.

09 — Fiabilité de l'IA (hallucinations)

Problème : L'IA invente des références, des API, des comportements ou des « bonnes pratiques » qui n'existent pas.

Risque : Code incorrect, failles de sécurité, décisions basées sur des informations fausses.

Ce qu'il faut faire : Toujours vérifier. Demander des sources, des exemples testables, et tester soi-même. Pour les domaines critiques (légal, médical, financier), ne jamais faire confiance à une réponse non vérifiée. L'IA est un accélérateur, pas une source de vérité.

10 — Systèmes distribués & théorème CAP

Problème : Deux serveurs (ou deux bases) divergent : l'un a une version des données, l'autre en a une autre (partition réseau).

Risque : Deux « vérités » différentes selon le serveur contacté.

Ce qu'il faut faire : Comprendre le théorème CAP : on ne peut pas avoir à la fois Cohérence, Disponibilité et Tolérance aux partitions. Choisir explicitement (souvent : cohérence éventuelle + compensation). Documenter le choix.

Ces 10 défis ne sont pas théoriques. Ils arrivent dans les applications réelles. Quand tu demandes quelque chose à l'IA, tu peux maintenant poser des questions précises : « Comment garantir l'idempotence ici ? », « Que se passe-t-il en cas de double clic ? », « Est-ce que cette vérification de permission empêche l'IDOR ? ».

PARTIE 3 — CHECKLIST UNIVERSELLE AVANT DE DÉMARRER

Web ou Mobile • Idée trouvée sur un réseau • Projet perso ou business. Les recherches et étapes obligatoires pour ne pas partir dans le vide.

1. Clarifier l'idée (obligatoire)

Avant toute recherche, tu dois pouvoir répondre clairement à ces questions. Si tu ne peux pas, l'idée n'est pas encore prête.

- ■ Quel problème précis cette application résout-elle ?
- ■ Qui a ce problème le plus souvent ? (cible exacte)
- ■ Comment ces personnes résolvent-elles le problème aujourd'hui ?
- ■ Pourquoi ta solution serait-elle meilleure ou plus simple ?
- ■ Quelle est la valeur principale que l'utilisateur gagne ?
- ■ Est-ce un besoin fréquent ou rare ?

Si tu ne peux pas répondre en 2-3 phrases claires → l'idée est encore trop vague.

2. Recherche sur le problème et le marché

- ■ Le problème est-il réellement ressenti par beaucoup de gens ?
- ■ Existe-t-il des discussions, plaintes ou demandes sur ce sujet (forums, Reddit, Twitter/X, groupes Facebook…) ?
- ■ Quelle est la taille approximative du marché ou de l'audience ?
- ■ Y a-t-il des tendances (croissance, réglementation, nouvelles habitudes) qui favorisent la solution ?

3. Analyse des concurrents

- ■ Qui propose déjà une solution (même imparfaite) ?
- ■ Quels sont leurs points forts et leurs points faibles (avis utilisateurs) ?
- ■ Qu'est-ce qu'ils ne font pas (ou mal) et que tu pourrais faire mieux ?
- ■ Comment te différencier sans forcément « révolutionner » le marché ?

4. Validation de la cible

- ■ As-tu parlé à de vraies personnes de ta cible ?
- ■ As-tu testé l'intérêt (sondage, landing page, précommandes, discussions) ?
- ■ Les gens disent-ils « je l'utiliserais » ou « je paierais pour ça » ?

5. Faisabilité technique & légale

- ■ As-tu les compétences (ou l'accès à l'IA + la capacité de valider) pour construire le MVP ?
- ■ Y a-t-il des contraintes légales fortes (santé, finance, enfants, données personnelles…) ?
- ■ Faut-il des conditions d'utilisation et une politique de confidentialité ?
- ■ Y a-t-il des réglementations spécifiques à ton secteur (RGPD, etc.) ?
- ■ As-tu prévu les bases de sécurité (auth, HTTPS, validation, permissions, rate limiting) ?

6. Définition du MVP

- ■ Peux-tu décrire le MVP en une phrase claire ?
- ■ As-tu listé les 3 à 5 fonctionnalités vraiment indispensables (must-have) ?
- ■ As-tu clairement séparé must-have vs nice-to-have ?
- ■ Le parcours utilisateur principal est-il décrit étape par étape ?

7. Préparation avant de coder

- ■ Cahier des charges ou document de cadrage simple rédigé
- ■ Stack technique choisie (et justifiée)
- ■ Outils d'organisation prêts (Notion, GitHub, etc.)
- ■ Checklist sécurité relue et adaptée au projet
- ■ Les 10 défis d'ingénierie ci-dessus ont été parcourus : lesquels s'appliquent déjà à ton MVP ?

8. Ordre recommandé à suivre

Étape	Action	Résultat attendu
1	Clarifier l'idée par écrit	Problème + cible + valeur clairs

2	Rechercher le problème & le marché	Preuve que le besoin existe
3	Analyser les concurrents	Écart / opportunité identifiée
4	Parler à la cible	Validation humaine
5	Définir le MVP	Liste courte de fonctionnalités
6	Vérifier faisabilité + légal + sécurité	Go / No-Go réaliste
7	Rédiger le cadrage	Document de référence
8	Seulement ensuite → développer	Construction utile

Règle finale : Une idée qui n'a pas passé ces étapes a de fortes chances de faire perdre du temps. Ce document est ta checklist permanente. Utilise-la à chaque nouvelle idée, quel que soit le type d'application.

PARTIE 4 — RÉSUMÉ EXÉCUTIF & POSTURE PERMANENTE

Ce que tu dois toujours garder en tête

1. **Sécurité dès le jour 1** — pas à la fin.
2. **Validation côté serveur + permissions** — ne jamais faire confiance au frontend.
3. **Rate limiting** sur tout ce qui peut être abusé (login, formulaires, API).
4. **Idempotence** sur les actions critiques (paiement, création, envoi).
5. **Tester les cas limites** : double-clic, URL modifiée, données invalides, offline, plantage.
6. **L'IA accélère, mais ne remplace pas ton jugement** — surtout sur la sécurité et les cas limites.
7. **Documenter les décisions importantes** pour pouvoir maintenir plus tard.
8. **Relire ce guide** avant chaque fonctionnalité sensible ou chaque nouvelle idée.

Phrase de référence

« Je ne fais pas confiance au frontend. Tout ce qui est important est vérifié et protégé côté serveur. Je demande à l'IA d'expliquer les mesures de sécurité et les cas limites. Je reste le décideur final. »

Ce document regroupe : la sécurité progressive (Junior → Senior), le rate limiting, les 10 défis d'ingénierie réels, et la checklist universelle avant de démarrer. Il est conçu pour rester utile même dans 2 ou 3 ans. Relis-le régulièrement. Mets-le à jour si tu découvres de nouvelles bonnes pratiques.

Sécurité + Solidité technique = responsabilité du développeur, pas de l'IA.

— Fin du guide —
À conserver et à relire avant chaque nouvelle fonctionnalité ou idée.